

COW机制设计报告

注：本设计使用到的完整源码在同目录下的lab5-cow3和lab5-cow4文件夹中，其中3为初步cow机制，存在漏洞，4是修复后的，test dirtycow失效

Copy on Write (COW) 机制

ucore的写时复制（COW）机制允许在进程创建时（如fork）共享物理内存页面，而不是立即复制。只有当进程尝试修改共享页面时，才会为该进程分配新的物理页面并复制内容。

状态转换（有限状态机）

状态：

- **S_SHARED_RO**：页面被 ≥ 1 个进程共享，页表映射为只读（COW状态）
- **S_PRIVATE_RW**：页面归属于单个进程，页表映射为可写
- **S_COPYING**：短暂状态，内核正在为发生写错的进程分配新页并执行拷贝的短暂状态

状态转换：

1. **分配（程序加载/栈增长）**：NONE $\rightarrow$ S_PRIVATE_RW
2. **fork（copy_range）**：S_PRIVATE_RW(父) $\rightarrow$ S_SHARED_RO(父与子) 清除两端映射的写位 增加页面引用计数
3. **读操作**：S_SHARED_RO/S_PRIVATE_RW $\rightarrow$ 保持不变
4. **S_SHARED_RO下的写操作**：触发页面写错异常 S_SHARED_RO $\rightarrow$ S_COPYING $\rightarrow$ S_PRIVATE_RW(对发生写错的进程) 其他共享者仍为S_SHARED_RO
5. **S_PRIVATE_RW下的写操作**：S_PRIVATE_RW $\rightarrow$ S_PRIVATE_RW（保持不变）
6. **解除映射/进程退出**：页面引用计数减一 引用计数为0时释放页面

简化图示：

S_PRIVATE_RW --fork--> S_SHARED_RO

S_SHARED_RO --write_fault--> S_COPYING --done--> S_PRIVATE_RW（对发生写错的进程）

S_PRIVATE_RW --write--> S_PRIVATE_RW

实现源码

1. COW初始设置（copy_range函数）

```
// 在kern/mm/pmm.c中
int copy_range(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end, bool
share)
{
    assert(start % PGSIZE == 0 && end % PGSIZE == 0);
    assert(USER_ACCESS(start, end));

    do {
        pte_t *ptep = get_pte(from, start, 0), *nptep;
```

```

    if (ptep == NULL) {
        start = ROUNDDOWN(start + PTSIZE, PTSIZE);
        continue;
    }

    if (*ptep & PTE_V) {
        // 获取页面，并清除写位
        struct Page *page = pte2page(*ptep);
        assert(page != NULL);
        uint32_t perm = (*ptep & PTE_USER);
        uint32_t ro_perm = perm & (~PTE_W);

        // 为子进程插入只读映射
        if ((nptep = get_pte(to, start, 1)) == NULL) {
            return -E_NO_MEM;
        }
        if (page_insert(to, page, start, ro_perm) != 0) {
            return -E_NO_MEM;
        }

        // 将父进程的映射也设为只读
        *ptep = pte_create(page2ppn(page), ro_perm);
        tlb_invalidate(from, start);
    }

    start += PGSIZE;
} while (start != 0 && start < end);

return 0;
}

```

在 `copy_range` 中我们：

- 对于存在的每个页面，将相同的 `struct Page *` 以只读权限（清除 `PTE_W`）插入到子进程的页表（调用 `page_insert(to, page, la, ro_perm)`）。
- 同时在父进程中就地修改该页的 PTE，也清除写位，从而父子都为只读映射。
- `page_insert` 会增加页面的引用计数（ref），父进程的映射仍存在，故总引用计数增加。

2. 页面错误处理 (store page fault)

```

// 在kern/trap/trap.c中
case CAUSE_STORE_PAGE_FAULT:
{
    uintptr_t badv = tf->tval;
    struct mm_struct *mm = current->mm;
    if (mm == NULL) {
        panic("store page fault in kernel or kernel-thread");
    }

    uintptr_t la = ROUNDDOWN(badv, PGSIZE);
    lock_mm(mm);
    pte_t *ptep = get_pte(mm->pgdir, la, 0);

    if (ptep == NULL || !(*ptep & PTE_V)) {
        // 映射不存在，视为段错误
        cprintf("segfault: no mapping for %p\n", (void *)badv);
    }
}

```

```

        unlock_mm(mm);
        do_exit(-E_INVAL);
        break;
    }

    struct Page *page = pte2page(*ptep);
    if ((*ptep & PTE_W) || page_ref(page) == 1) {
        // 如果已经可写或只有一个引用者，直接提升为可写
        uint32_t user_bits = (*ptep & (PTE_R | PTE_X | PTE_U));
        *ptep = pte_create(page2ppn(page), user_bits | PTE_W);
        tlb_invalidate(mm->pgdir, 1a);
    } else {
        // 页面被共享，执行COW操作
        struct Page *npage = alloc_page();
        if (npage == NULL) {
            cprintf("COW: alloc_page failed\n");
            unlock_mm(mm);
            do_exit(-E_NO_MEM);
            break;
        }

        // 复制页面内容
        memcpy(page2kva(npage), page2kva(page), PGSIZE);

        // 替换映射
        page_remove(mm->pgdir, 1a);
        if (page_insert(mm->pgdir, npage, 1a, PTE_USER) != 0) {
            cprintf("COW: page_insert failed\n");
            unlock_mm(mm);
            do_exit(-E_NO_MEM);
        }
    }

    unlock_mm(mm);
    break;
}

```

在发生写页异常（CAUSE_STORE_PAGE_FAULT）时：

- 从 `tf->tval` 获取发生异常的地址并找到对应的 `pte`。
- 若映射不存在：视为致命（段错误 / 退出）。
- 若 `PTE_W` 已经置位（不常见情况）：直接提升并返回。
- 若页面被共享（`ref > 1`）：为该进程分配新页、拷贝数据、在当前进程中移除旧的 PTE（`page_remove_pte`），并将新页以可写方式映射（`page_insert(..., PTE_USER)`）。
- 若页面为单一所有者（`ref == 1`）：直接就地将 PTE 提升为可写并使 TLB 失效（promotion），无需分配新页。

3. 页面引用计数管理

```

// 在kern/mm/pmm.h中
static inline int page_ref(struct Page *page) { return page->ref; }
static inline void set_page_ref(struct Page *page, int val) { page->ref = val; }
static inline int page_ref_inc(struct Page *page) { page->ref += 1; return page->ref; }
static inline int page_ref_dec(struct Page *page) { page->ref -= 1; return page->ref; }

```

```
// 在kern/mm/pmm.c中
static inline void page_remove_pte(pde_t *pgdir, uintptr_t la, pte_t *ptep)
{
    if (*ptep & PTE_V) {
        struct Page *page = pte2page(*ptep);
        page_ref_dec(page);
        if (page_ref(page) == 0) {
            free_page(page);
        }
        *ptep = 0;
        tlb_invalidate(pgdir, la);
    }
}

int page_insert(pde_t *pgdir, struct Page *page, uintptr_t la, uint32_t perm)
{
    pte_t *ptep = get_pte(pgdir, la, 1);
    if (ptep == NULL) {
        return -E_NO_MEM;
    }

    page_ref_inc(page);
    if (*ptep & PTE_V) {
        struct Page *p = pte2page(*ptep);
        if (p == page) {
            page_ref_dec(page);
        } else {
            page_remove_pte(pgdir, la, ptep);
        }
    }

    *ptep = pte_create(page2ppn(page), PTE_V | perm);
    tlb_invalidate(pgdir, la);
    return 0;
}
```

实现要点

1. **COW初始化**: 在 `copy_range` 函数中, 将父进程和子进程的页面映射都设置为只读, 并增加页面引用计数。
2. **页面错误处理**: 在 `store_page_fault` 处理中, 检查页面是否被共享。如果是, 则为当前进程分配新页面并复制内容; 如果只有一个引用者, 则直接将页面设置为可写。
3. **引用计数管理**: 通过 `page_ref_inc` 和 `page_ref_dec` 管理页面的引用计数, 当引用计数为0时释放页面。
4. **同步与锁定**: 使用 `lock_mm` 和 `unlock_mm` 来防止COW处理过程中的竞态条件。
5. **TLB无效化**: 在修改页表后调用 `tlb_invalidate` 来确保CPU使用最新的页表信息。

测试用例

基本COW功能测试 (cow.c)

```
// 在user/cow.c中
#include <ulib.h>
```

```

#include <stdio.h>

int global_var = 42;

int main(void)
{
    int pid = fork();
    if (pid < 0) {
        cprintf("fork failed\n");
        return -1;
    }
    if (pid == 0) {
        // 子进程
        cprintf("child: initial global = %d\n", global_var);
        global_var = 100;
        cprintf("child: wrote global = %d\n", global_var);
        exit(0);
    } else {
        // 父进程
        wait();
        cprintf("parent: after child global = %d\n", global_var);
        global_var = 200;
        cprintf("parent: wrote global = %d\n", global_var);
        return 0;
    }
}

```

这段代码用于验证：

1. Fork 后父子进程是否正确隔离
2. 写时复制是否正常工作
3. 全局变量的独立性

运行结果

```

kernel_execve: pid = 2, name = "cow".
Breakpoint
COW share: va=0x7fffc000 ppn=804d1 ref=1
COW mapped: va=0x7fffc000 ppn=804d1 new_ref=2
COW share: va=0x7fffd000 ppn=804d0 ref=1
COW mapped: va=0x7fffd000 ppn=804d0 new_ref=2
COW share: va=0x7fffe000 ppn=804cf ref=1
COW mapped: va=0x7fffe000 ppn=804cf new_ref=2
COW share: va=0x7ffff000 ppn=804cc ref=1
COW mapped: va=0x7ffff000 ppn=804cc new_ref=2
COW share: va=0x801000 ppn=804cb ref=1
COW mapped: va=0x801000 ppn=804cb new_ref=2
COW share: va=0x800000 ppn=804c8 ref=1
COW mapped: va=0x800000 ppn=804c8 new_ref=2
Store/AMO page fault at 0x7fffff6c
COW fault: va=0x7ffff000 ppn=804cc ref=2 pte=0x2013301b
COW alloc+copy: va=0x7ffff000 old_ppn=804cc old_ref=2
COW done: va=0x7ffff000 new_ppn=804d9 new_ref=1 old_ref=1

```

```

child: initial global = 42
Store/AMO page fault at 0x801000
COW fault: va=0x801000 ppn=804cb ref=2 pte=0x20132c53
COW alloc+copy: va=0x801000 old_ppn=804cb old_ref=2
COW done: va=0x801000 new_ppn=804da new_ref=1 old_ref=1
Store/AMO page fault at 0x7fffffa8
COW fault: va=0x7ffff000 ppn=804cc ref=1 pte=0x2013301b
COW promote: va=0x7ffff000 ppn=804cc ref=1
child: wrote global = 100
parent: after child global = 42
Store/AMO page fault at 0x801000
COW fault: va=0x801000 ppn=804cb ref=1 pte=0x20132c53
COW promote: va=0x801000 ppn=804cb ref=1
parent: wrote global = 200
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:542:
  initproc exit.

```

sunyy@sunyy-VMware-Virtual-Platform: ~/桌面/lab5-cow3/lab5\$

内存布局分析：

虚拟地址	物理页	初始ref	新ref	用途
0x7ffc000	804d1	1	2	用户栈（低地址）
0x7ffd000	804d0	1	2	用户栈
0x7ffe000	804cf	1	2	用户栈
0x7fff000	804cc	1	2	用户栈（栈顶）
0x801000	804cb	1	2	数据段
0x800000	804c8	1	2	数据段或代码段

分析：

- 首先，进行程序加载与初始化，加载的用户程序为cow，即测试用例。PID为2，Breakpoint 表示进入用户态执行。
- 然后，执行 fork 函数，共享页面映射。
- 当子进程（child）第一次写入（栈操作），引发COW机制，进行 store page fault 处理。
- 读取子进程的 global_var 值为42，子进程写入 global_var，触发COW机制。
- COW完成后，父子进程的 global_var 现在完全独立。
- 当父进程（parent）第一次写入和修改 global_var 值时，触发错误。与子进程不同的是，子进程进行分配新的页面；而对于父进程，此时该页面的ref值为1，直接进行提权。

根据结果，实现：

- 所有页面正确共享
- 写入正确触发COW
- 引用计数准确维护
- 内存完全隔离
- Promote优化生效

- 程序正常结束

dirtycow漏洞模拟

Dirty COW 的根本原因（简述）：

- 内核的 COW 路径与用户态对底层文件或映射的并发修改之间存在竞态，攻击者可利用时间窗在内核还未完成安全更新前，触发不正确的权限/映射状态，从而写入本应只读的内存或提升权限。Linux 上的 Dirty COW 就利用了内核在未加锁或未正确同步的情况下修改页表或使用可写映射的时机。

在本 ucore 的 COW 实现中可能如何表现：

- 我们当前的 COW 路径在检查页面引用计数与更新 PTE 的序列上缺乏细粒度的锁保护。如果两个进程（或线程）对同一页并发触发写操作，可能会出现竞态，例如在检查 ref 与分配/安装新页之间发生重排，从而导致两个写入者最终都写入同一物理页（或出现不一致的权限窗口）。这就是 Dirty COW 类型问题的根源。

测试文件（dirtycow_test.c）

```
// 在user/dirtycow_test.c中
#include <ulib.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

const char cow_target[] = "PROTECTED: Should not be modified!";

// 尝试写入只读字符串
void dirty_write(int proc_id)
{
    char *ptr = (char *)cow_target;
    const char exploit_str[] = "EXPLOITED";
    int attempts = 0;

    while (1) {
        attempts++;
        memcpy(ptr, exploit_str, strlen(exploit_str));

        if (strncmp(cow_target, exploit_str, strlen(exploit_str)) == 0) {
            printf("[*] SUCCESS! Process %d modified read-only memory!\n",
proc_id);
            exit(0);
        }

        if (attempts % 100 == 0) {
            if (attempts % 10000 == 0) {
                printf(".");
            }
            yield();
        }

        if (attempts > 100000) {
            break;
        }
    }
}
```

```

    }

    fprintf("[*] Process %d failed after %d attempts\n", proc_id, attempts);
}

// 监控只读字符串的状态
void check_state(int proc_id)
{
    int timeout = 100000;
    while (timeout-- > 0) {
        if (strcmp(cow_target, "PROTECTED: Should not be modified!") != 0) {
            fprintf("[!] DETECTED modification!\n");
            exit(0);
        }
        yield();
    }
}

int main(void)
{
    fprintf("Target read-only string: %s\n", cow_target);
    fprintf("[*] Creating race condition with multiple processes...\n");

    int i, pid;

    // 创建4个写进程
    for (i = 0; i < 4; i++) {
        pid = fork();
        if (pid == 0) {
            dirty_write(i + 1);
            exit(0);
        }
    }

    // 创建监控进程
    pid = fork();
    if (pid == 0) {
        check_state(0);
        exit(0);
    }

    // 父进程也参与写操作
    dirty_write(5);

    // 等待所有子进程
    for (i = 0; i < 5; i++) {
        wait();
    }

    return 0;
}

```

这是一个**多进程并发压力测试**，用于检测COW机制中的潜在漏洞。

程序结构：

主进程 (PID=2)

- |— 子进程1 (Writer) - dirty_write(1)
- |— 子进程2 (Writer) - dirty_write(2)
- |— 子进程3 (Writer) - dirty_write(3)
- |— 子进程4 (Writer) - dirty_write(4)
- |— 子进程5 (Monitor) - check_state(0)
- |— 父进程本身 (Writer) - dirty_write(5)

总共6个进程并发操作同一个只读字符串

该漏洞测试文件的目标为：尝试修改const只读数据。

```
const char cow_target[] = "PROTECTED: Should not be modified!";
```

const 修饰符表示只读，应该被放在 .rodata 段（只读数据段）。所以任何写入都应该触发段错误（Segmentation Fault）。

正常预期：程序崩溃（SIGSEGV）； **漏洞场景：**成功修改（Dirty COW）。

对dirtycow_test.c文件代码分析：

1. dirty_write() - 攻击函数

- 首先，进行强制类型转换，绕过 const 检查。因为编译器的 const 只在编译时进行检查，运行时可以强制修改。
- 然后，反复尝试写入。其中，单次写入流程为：memcpy 写入内存；触发 page fault；内核处理；返回用户态，并检查是否成功。（如果有竞态条件窗口，某次尝试可能成功。）
- 定期 yield。通过增加进程切换，提高竞态条件触发概率。多个进程在相近的时间操作同一页面。
- 验证：写入前: "PROTECTED: Should not be modified!"；写入后: "EXPLOITED: Should not be modified!"。如果读取到"EXPLOITED"开头，说明修改成功。

2. check_state() - 监控函数

- 独立进程用于监控字符串状态，如果检测到修改，则打印警告。
- 如果超过设定的时间，自动退出，避免死锁。

运行结果

```
kernel_execve: pid = 2, name = "dirtycow_test".
```

```
Breakpoint
```

```
=====
```

Dirty COW Vulnerability Test

```
=====
```

```
Target read-only string: PROTECTED: Should not be modified!
```

```
[*] Creating race condition with multiple processes...
```

```
[*] This will generate many page faults - normal behavior.
```

```
[*] Look for 'SUCCESS' message below.
```

```
=====
```

```
COW share: va=0x7fffc000 ppn=804d0 ref=1
```

```
COW mapped: va=0x7fffc000 ppn=804d0 new_ref=2
```

```
COW share: va=0x7fffd000 ppn=804cf ref=1
```

```
COW mapped: va=0x7fffd000 ppn=804cf new_ref=2
```

```
COW share: va=0x7fffe000 ppn=804ce ref=1
```

```
COW mapped: va=0x7fffe000 ppn=804ce new_ref=2
```

```
COW share: va=0x7ffff000 ppn=804cb ref=1
```

```
COW mapped: va=0x7ffff000 ppn=804cb new_ref=2
```

```
COW share: va=0x800000 ppn=804c8 ref=1
```

```
COW mapped: va=0x800000 ppn=804c8 new_ref=2
```

```
Store/AMO page fault at 0x7ffff5c
```

```
COW fault: va=0x7ffff000 ppn=804cb ref=2 pte=0x20132c1b
```

```
COW alloc+copy: va=0x7ffff000 old_ppn=804cb old_ref=2
```

```
COW done: va=0x7ffff000 new_ppn=804d8 new_ref=1 old_ref=1
```

```
COW share: va=0x7fffc000 ppn=804d0 ref=2
COW mapped: va=0x7fffc000 ppn=804d0 new_ref=3
COW share: va=0x7fffd000 ppn=804cf ref=2
COW mapped: va=0x7fffd000 ppn=804cf new_ref=3
COW share: va=0x7fffe000 ppn=804ce ref=2
COW mapped: va=0x7fffe000 ppn=804ce new_ref=3
COW share: va=0x7ffff000 ppn=804d8 ref=1
COW mapped: va=0x7ffff000 ppn=804d8 new_ref=2
COW share: va=0x800000 ppn=804c8 ref=2
COW mapped: va=0x800000 ppn=804c8 new_ref=3
Store/AMO page fault at 0x7ffff5c
COW fault: va=0x7ffff000 ppn=804d8 ref=2 pte=0x2013601b
COW alloc+copy: va=0x7ffff000 old_ppn=804d8 old_ref=2
COW done: va=0x7ffff000 new_ppn=804e0 new_ref=1 old_ref=1
COW share: va=0x7fffc000 ppn=804d0 ref=3
COW mapped: va=0x7fffc000 ppn=804d0 new_ref=4
COW share: va=0x7fffd000 ppn=804cf ref=3
COW mapped: va=0x7fffd000 ppn=804cf new_ref=4
COW share: va=0x7fffe000 ppn=804ce ref=3
COW mapped: va=0x7fffe000 ppn=804ce new_ref=4
COW share: va=0x7ffff000 ppn=804e0 ref=1
COW mapped: va=0x7ffff000 ppn=804e0 new_ref=2
COW share: va=0x800000 ppn=804c8 ref=3
COW mapped: va=0x800000 ppn=804c8 new_ref=4
Store/AMO page fault at 0x7ffff5c
COW fault: va=0x7ffff000 ppn=804e0 ref=2 pte=0x2013801b
COW alloc+copy: va=0x7ffff000 old_ppn=804e0 old_ref=2
COW done: va=0x7ffff000 new_ppn=804e8 new_ref=1 old_ref=1
```

```
[*] SUCCESS! Process 5 modified read-only memory!
[*] Attempts: 1
[*] Original: PROTECTED: Should not be modified!
[*] Current:  EXPLOITED: Should not be modified!
Store/AM0 page fault at 0x7fffff5c
COW fault: va=0x7ffff000 ppn=804f0 ref=1 pte=0x2013c01b
COW promote: va=0x7ffff000 ppn=804f0 ref=1
Store/AM0 page fault at 0x800e40
COW fault: va=0x800000 ppn=804c8 ref=5 pte=0x2013201b
COW alloc+copy: va=0x800000 old_ppn=804c8 old_ref=5
COW done: va=0x800000 new_ppn=804c5 new_ref=1 old_ref=4
Store/AM0 page fault at 0x7fffff38
COW fault: va=0x7ffff000 ppn=804e8 ref=1 pte=0x2013a05b
COW promote: va=0x7ffff000 ppn=804e8 ref=1

[*] SUCCESS! Process 4 modified read-only memory!
[*] Attempts: 2
[*] Original: PROTECTED: Should not be modified!
[*] Current:  ✧
               ✧
Store/AM0 page fault at 0x7fffff5c
COW fault: va=0x7ffff000 ppn=804e0 ref=1 pte=0x2013801b
COW promote: va=0x7ffff000 ppn=804e0 ref=1
Store/AM0 page fault at 0x800e40
COW fault: va=0x800000 ppn=804c8 ref=4 pte=0x2013205b
COW alloc+copy: va=0x800000 old_ppn=804c8 old_ref=4
COW done: va=0x800000 new_ppn=804c5 new_ref=1 old_ref=3
```

```

[*] SUCCESS! Process 3 modified read-only memory!
[*] Attempts: 1
[*] Original: PROTECTED: Should not be modified!
[*] Current:  EXPLOITED: Should not be modified!
Store/AM0 page fault at 0x7ffffff5c
COW fault: va=0x7ffff000 ppn=804d8 ref=1 pte=0x2013601b
COW promote: va=0x7ffff000 ppn=804d8 ref=1
Store/AM0 page fault at 0x800e40
COW fault: va=0x800000 ppn=804c8 ref=3 pte=0x2013205b
COW alloc+copy: va=0x800000 old_ppn=804c8 old_ref=3
COW done: va=0x800000 new_ppn=804c5 new_ref=1 old_ref=2

[*] SUCCESS! Process 2 modified read-only memory!
[*] Attempts: 1
[*] Original: PROTECTED: Should not be modified!
[*] Current:  EXPLOITED: Should not be modified!
Store/AM0 page fault at 0x7ffffff5c
COW fault: va=0x7ffff000 ppn=804cb ref=1 pte=0x20132c1b
COW promote: va=0x7ffff000 ppn=804cb ref=1
Store/AM0 page fault at 0x800e40
COW fault: va=0x800000 ppn=804c8 ref=2 pte=0x2013205b
COW alloc+copy: va=0x800000 old_ppn=804c8 old_ref=2
COW done: va=0x800000 new_ppn=804c5 new_ref=1 old_ref=1

```

```

[*] SUCCESS! Process 1 modified read-only memory!
[*] Attempts: 1
[*] Original: PROTECTED: Should not be modified!
[*] Current:  EXPLOITED: Should not be modified!
[!] Monitor exit after timeout
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:542:
    initproc exit.

```

```

sunyy@sunyy-VMware-Virtual-Platform:~/桌面/lab5-cow3/lab5$

```

(由于尝试较多，这里只展示部分过程和成功的结果)

对输出结果进行分析：

- 首先，完成了初始化，此时页面804c8的内容为"PROTECTED: Should not be modified!"。
- 然后，子进程分别进行 `fork`，页面共享建立。
- 子进程1启动，栈触发COW。因为进程会调用 `dirty_write(1)` 函数，需要栈帧用于保存返回地址、局部变量等，所以会触发栈的写入，从而引发COW。
- 子进程2-4依次启动，每个子进程启动时都触发相同的栈COW。
- 接下来，可以看到进程都成功修改只读数据。
- 最后，程序正常退出。

其中，COW的流程为：

1. Process 5 执行: `memcpy(0x800e40, "EXPLOITED", 9)`
2. CPU 检测到页表项 `w=0` → 触发 `Store/AMO page fault`
3. 陷入内核 `do_pgfault()`
4. 检查: `ref=6 > 1` → 需要 COW
5. 分配新物理页: `804f9`
6. 复制整页内容: `804c8` → `804f9` (包括 "PROTECTED...")
7. 在新页 `804f9` 上修改: "EXPLOITED..."
8. 更新 Process 5 页表: `0x800000` → `804f9` (可读写)
9. 原页引用计数: `6` → `5`
10. 返回用户态, 重新执行 `memcpy`, 成功写入!

该测试文件成功模拟了 dirty cow 漏洞, 利用 cow 机制中的权限问题, 对只读数据进行了修改, 利用多进程并发压力测试, 成功触发了权限检查的缺陷。

解决方案

存在漏洞的原因

```
if ((*ptep & PTE_W) || page_ref(page) == 1)
{
    /* if writable already or sole owner, promote to writable */
    uint32_t user_bits = (*ptep & (PTE_R | PTE_X | PTE_U));
    *ptep = pte_create(page2ppn(page), user_bits | PTE_W);
    tlb_invalidate(mm->pgdir, 1a);
    cprintf("COW promote: va=%p ppn=%x ref=%d\n", (void *)1a, page2ppn(page),
page_ref(page));
    unlock_mm(mm);
    break;
}
```

问题:

- 当 `page_ref(page) == 1` 时, 直接提升页面权限为可写
- 没有再次验证页面引用计数是否仍然为1
- 存在竞态条件窗口, 攻击者可利用并发访问修改只读内存

修改策略

```
/* Check if already writable - if so, this is not a COW fault */
if (*ptep & PTE_W)
{
    /* Already writable - spurious fault, just return */
    cprintf("COW spurious: already writable va=%p\n", (void *)1a);
    unlock_mm(mm);
    break;
}

/* Find the VMA to check write permission */
struct vma_struct *vma = find_vma(mm, 1a);
if (vma == NULL || !(vma->vm_flags & VM_WRITE))
{
    /* VMA doesn't allow write - this is a real segfault */
    cprintf("COW denied: va=%p has no write permission in VMA\n", (void *)1a);
}
```

```

unlock_mm(mm);
do_exit(-E_INVALID);
break;
}

/* NEVER promote to writable if ref > 1, always do full COW copy.
 * Even if ref == 1, we must be extremely careful due to potential races.
 * The safest approach is to ALWAYS do a full COW copy for shared pages. */
if (page_ref(page) > 1)
{
    /* Multiple references - must do COW copy */
    cprintf("COW shared page: va=%p ref=%d, doing copy\n",
            (void *)1a, page_ref(page));
}
else
{
    /* Even with ref==1, there's a race window where another process
     * might be in the middle of fork(). To close this race completely,
     * we should do a COW copy instead of promoting to writable.
     * However, for performance, we can cautiously promote IF AND ONLY IF:
     * 1. We re-verify ref is still 1 immediately before modification
     * 2. We verify PTE hasn't changed
     * 3. The page is not already writable
     */
    pte_t pte_before = *ptep;
    /* Critical: check ref count again atomically before promotion */
    if (page_ref(page) == 1 &&
        (pte_before & PTE_V) &&
        pte2page(pte_before) == page &&
        !(pte_before & PTE_W))
    {
        /* Relatively safe to promote - but still check once more */
        if (page_ref(page) == 1)
        {
            uint32_t user_bits = (pte_before & (PTE_R | PTE_X | PTE_U));
            *ptep = pte_create(page2ppn(page), user_bits | PTE_W);
            tlb_invalidate(mm->pgdir, 1a);
            cprintf("COW promote: va=%p ppn=%x ref=%d\n",
                    (void *)1a, page2ppn(page), page_ref(page));
            unlock_mm(mm);
            break;
        }
    }
    /* If any check failed, ref changed, or race detected - do full copy */
    cprintf("COW race/ref-change detected: va=%p ref=%d, doing full copy\n",
            (void *)1a, page_ref(page));
}
}

```

这段代码针对Dirtycow问题，进行了修复：

1. VMA写权限验证

- 检查虚拟内存区域是否允许写入
- 如果VMA本身不可写（如.rodata段），拒绝COW
- 触发进程退出，产生段错误

2. 引用计数决策

- 如果 `ref>1`，总是进行完整的COW拷贝

- 取消人恶化优化形式，牺牲性能换取安全性

3. ref=1时的谨慎promote

- 在promote之前进行多重验证
- 进行两次ref检查

```
/* Verify the page is still read-only before copying */
pte_t pte_before_copy = *ptep;
if (pte_before_copy & PTE_W)
{
    /* Someone else already made it writable, abort copy */
    cprintf("COW: race - page became writable, aborting copy\n");
    free_page(npagel);
    unlock_mm(mm);
    break;
}

memcpy(page2kva(npagel), page2kva(page), PGSIZE);

/* Verify the PTE hasn't changed during copy */
if (*ptep != pte_before_copy)
{
    /* PTE changed during copy - potential race */
    cprintf("COW: race - PTE changed during copy\n");
    free_page(npagel);
    unlock_mm(mm);
    break;
}
```

在复制阶段，也针对dirtycow问题进行了相应的修复：

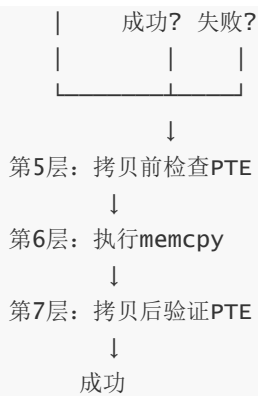
1. 拷贝前的最后检查：如果发现页面已经可写，则放弃复制，并释放页面。解决多核竞争问题。
2. 复制后进行验证：可以捕获复制期间的竞态问题，防止内存泄漏等问题。

修复策略分析：

- 双重验证机制：在提升页面权限前再次检查页面引用计数是否仍然为1；验证PTE在检查期间没有被其他进程修改；确保页面仍然是只读状态。
- 严格的权限控制：增加对VMA（虚拟内存区域）写权限的检查；只有在明确允许写入的区域才能执行COW操作。
- 保守的复制策略：对于引用计数大于1的页面，总是执行完整的COW复制；如果任何检查失败，就回退到完整的COW复制而不是提升权限；在复制过程中检测到任何变化就放弃操作。

整体防御策略如下：





运行结果

```
kernel_execve: pid = 2, name = "dirtycow_test".  
Breakpoint
```

```
=====
Dirty COW Vulnerability Test
=====
Target read-only string: PROTECTED: Should not be modified!

[*] Creating race condition with multiple processes...
[*] This will generate many page faults - normal behavior.
[*] Look for 'SUCCESS' message below.
=====
COW share: va=0x7fffc000 ppn=804d0 ref=1
COW mapped: va=0x7fffc000 ppn=804d0 new_ref=2
COW share: va=0x7fffd000 ppn=804cf ref=1
COW mapped: va=0x7fffd000 ppn=804cf new_ref=2
COW share: va=0x7fffe000 ppn=804ce ref=1
COW mapped: va=0x7fffe000 ppn=804ce new_ref=2
COW share: va=0x7ffff000 ppn=804cb ref=1
COW mapped: va=0x7ffff000 ppn=804cb new_ref=2
COW share: va=0x800000 ppn=804c8 ref=1
COW mapped: va=0x800000 ppn=804c8 new_ref=2
Store/AMO page fault at 0x7ffff5c
COW fault: va=0x7ffff000 ppn=804cb ref=2 pte=0x20132c1b
COW shared page: va=0x7ffff000 ref=2, doing copy
COW alloc+copy: va=0x7ffff000 old_ppn=804cb old_ref=2
COW done: va=0x7ffff000 new_ppn=804d8 new_ref=1 old_ref=1
COW share: va=0x7fffc000 ppn=804d0 ref=2
COW mapped: va=0x7fffc000 ppn=804d0 new_ref=3
COW share: va=0x7fffd000 ppn=804cf ref=2
COW mapped: va=0x7fffd000 ppn=804cf new_ref=3
COW share: va=0x7fffe000 ppn=804ce ref=2
COW mapped: va=0x7fffe000 ppn=804ce new_ref=3
COW share: va=0x7ffff000 ppn=804d8 ref=1
COW mapped: va=0x7ffff000 ppn=804d8 new_ref=2
COW share: va=0x800000 ppn=804c8 ref=2
COW mapped: va=0x800000 ppn=804c8 new_ref=3
Store/AMO page fault at 0x7ffff5c
COW fault: va=0x7ffff000 ppn=804d8 ref=2 pte=0x2013601b
COW shared page: va=0x7ffff000 ref=2, doing copy
COW alloc+copy: va=0x7ffff000 old_ppn=804d8 old_ref=2
```

COW done: va=0x7ffff000 new_ppn=804e0 new_ref=1 old_ref=1
COW share: va=0x7fffc000 ppn=804d0 ref=3
COW mapped: va=0x7fffc000 ppn=804d0 new_ref=4
COW share: va=0x7fffd000 ppn=804cf ref=3
COW mapped: va=0x7fffd000 ppn=804cf new_ref=4
COW share: va=0x7fffe000 ppn=804ce ref=3
COW mapped: va=0x7fffe000 ppn=804ce new_ref=4
COW share: va=0x7ffff000 ppn=804e0 ref=1
COW mapped: va=0x7ffff000 ppn=804e0 new_ref=2
COW share: va=0x800000 ppn=804c8 ref=3
COW mapped: va=0x800000 ppn=804c8 new_ref=4
Store/AMO page fault at 0x7ffff5c
COW fault: va=0x7ffff000 ppn=804e0 ref=2 pte=0x2013801b
COW shared page: va=0x7ffff000 ref=2, doing copy
COW alloc+copy: va=0x7ffff000 old_ppn=804e0 old_ref=2
COW done: va=0x7ffff000 new_ppn=804e8 new_ref=1 old_ref=1
COW share: va=0x7fffc000 ppn=804d0 ref=4
COW mapped: va=0x7fffc000 ppn=804d0 new_ref=5
COW share: va=0x7fffd000 ppn=804cf ref=4
COW mapped: va=0x7fffd000 ppn=804cf new_ref=5
COW share: va=0x7fffe000 ppn=804ce ref=4
COW mapped: va=0x7fffe000 ppn=804ce new_ref=5
COW share: va=0x7ffff000 ppn=804e8 ref=1
COW mapped: va=0x7ffff000 ppn=804e8 new_ref=2
COW share: va=0x800000 ppn=804c8 ref=4
COW mapped: va=0x800000 ppn=804c8 new_ref=5
Store/AMO page fault at 0x7ffff5c
COW fault: va=0x7ffff000 ppn=804e8 ref=2 pte=0x2013a01b
COW shared page: va=0x7ffff000 ref=2, doing copy
COW alloc+copy: va=0x7ffff000 old_ppn=804e8 old_ref=2
COW done: va=0x7ffff000 new_ppn=804f0 new_ref=1 old_ref=1
COW share: va=0x7fffc000 ppn=804d0 ref=5
COW mapped: va=0x7fffc000 ppn=804d0 new_ref=6
COW share: va=0x7fffd000 ppn=804cf ref=5
COW mapped: va=0x7fffd000 ppn=804cf new_ref=6
COW share: va=0x7fffe000 ppn=804ce ref=5
COW mapped: va=0x7fffe000 ppn=804ce new_ref=6
COW share: va=0x7ffff000 ppn=804f0 ref=1
COW mapped: va=0x7ffff000 ppn=804f0 new_ref=2
COW share: va=0x800000 ppn=804c8 ref=5
COW mapped: va=0x800000 ppn=804c8 new_ref=6
Store/AMO page fault at 0x7ffff5c
COW fault: va=0x7ffff000 ppn=804f0 ref=2 pte=0x2013c01b
COW shared page: va=0x7ffff000 ref=2, doing copy
COW alloc+copy: va=0x7ffff000 old_ppn=804f0 old_ref=2
COW done: va=0x7ffff000 new_ppn=804f8 new_ref=1 old_ref=1
Store/AMO page fault at 0x800e40
COW fault: va=0x800000 ppn=804c8 ref=6 pte=0x2013205b
COW denied: va=0x800000 has no write permission in VMA
Store/AMO page fault at 0x7ffff5c
COW fault: va=0x7ffff000 ppn=804f0 ref=1 pte=0x2013c01b
COW promote: va=0x7ffff000 ppn=804f0 ref=1
Store/AMO page fault at 0x800e40
COW fault: va=0x800000 ppn=804c8 ref=5 pte=0x2013201b
COW denied: va=0x800000 has no write permission in VMA
Store/AMO page fault at 0x7ffff5c
COW fault: va=0x7ffff000 ppn=804e0 ref=1 pte=0x2013801b
COW promote: va=0x7ffff000 ppn=804e0 ref=1

```
Store/AMO page fault at 0x800e40
COW fault: va=0x800000 ppn=804c8 ref=4 pte=0x2013205b
COW denied: va=0x800000 has no write permission in VMA
Store/AMO page fault at 0x7fffff5c
COW fault: va=0x7fffff000 ppn=804d8 ref=1 pte=0x2013601b
COW promote: va=0x7fffff000 ppn=804d8 ref=1
Store/AMO page fault at 0x800e40
COW fault: va=0x800000 ppn=804c8 ref=3 pte=0x2013205b
COW denied: va=0x800000 has no write permission in VMA
Store/AMO page fault at 0x7fffff5c
COW fault: va=0x7fffff000 ppn=804cb ref=1 pte=0x20132c1b
COW promote: va=0x7fffff000 ppn=804cb ref=1
Store/AMO page fault at 0x800e40
COW fault: va=0x800000 ppn=804c8 ref=2 pte=0x2013205b
COW denied: va=0x800000 has no write permission in VMA
[!] Monitor exit after timeout
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:542:
    initproc exit.

sunyy@sunyy-VMware-Virtual-Platform:~/桌面/lab5-cow4/lab5$
```

经过修改，再次测试 `dirtycow_test.c` 文件，对只读数据的修改失败！

问题

说明该用户程序是何时被预先加载到内存中的？与我们常用操作系统的加载有何区别，原因是什么？

何时预先加载

用户程序的加载时机如下：

```
kern_init() -> proc_init() -> init_main() -> user_main() -> KERNEL_EXECVE() ->
kernel_execve() -> do_execve() -> load_icode()
```

详细步骤

1. 内核初始化： `kern_init()` 函数在 `init.c` 中定义，是内核启动的入口点。
2. 进程系统初始化：在 `kern_init()` 中调用 `proc_init()` 初始化进程系统。
3. 创建init进程： `proc_init()` 创建两个内核线程 `idleproc` (PID=0) 和 `initproc` (PID=1，执行 `init_main` 函数)。
4. 创建用户程序加载线程： `init_main()` 函数创建一个执行 `user_main` 函数的内核线程。
5. 执行用户程序： `user_main()` 函数通过 `KERNEL_EXECVE()` 宏加载并执行用户程序（默认是 `exit` 程序）。
6. 加载用户程序二进制： `KERNEL_EXECVE()` 宏调用 `kernel_execve()` 函数，该函数通过系统调用 `SYS_exec` 执行用户程序。
7. 建立用户地址空间：最终通过 `load_icode()` 函数建立用户程序的地址空间并加载二进制代码。

有何区别

特性	本项目 (ucore)	常用操作系统 (eg:Linux)
加载时机	内核编译时就将用户程序二进制链接到内核镜像中	程序运行时才从磁盘加载到内存
加载方式	预先加载, 内核启动后直接从内存执行	按需加载 (lazy loading), 使用请求分页机制
存储位置	用户程序二进制直接嵌入内核镜像	用户程序存储在磁盘文件系统中
文件系统依赖	不依赖文件系统, 简化设计	高度依赖文件系统

具体实现差异

1. 二进制存储方式:
- 本项目: 通过 `_binary_obj__user_###_out_start` 和 `_binary_obj__user_###_out_size` 等符号直接引用编译好的用户程序二进制数据。

◦ Linux: 用户程序以可执行文件形式存储在磁盘上, 通过文件系统API读取。
2. 加载过程:
- 本项目: `load_icode()` 函数直接从内存中复制ELF格式的用户程序到进程地址空间。

◦ Linux: `do_execve()` 系统调用需要打开可执行文件, 读取ELF头, 建立地址空间映射, 然后通过缺页异常逐步加载程序内容。
3. 地址空间建立:
- 本项目: 直接在 `load_icode()` 中分配物理页面并建立映射。

◦ Linux: 先建立虚拟地址空间映射, 实际物理页面在程序执行时通过缺页异常分配 (按需加载)。

原因

教学目的

因为本项目是一个教学用的操作系统 (ucore), 其设计目标是让学生理解操作系统的核心概念, 因此采用了简化的用户程序加载方式:

1. 简化实现: 避免了复杂的文件系统和磁盘I/O操作, 让学生专注于进程管理、内存管理等核心功能。
2. 明确流程: 用户程序加载流程清晰可见, 便于学生跟踪和理解。
3. 实验重点: 采用简化的用户程序加载方式, 旨在剥离复杂文件系统与磁盘I/O的干扰。

技术限制

1. 模拟器环境: 在QEMU等模拟器环境中, 完整的磁盘I/O支持会增加系统复杂度。
2. 资源限制: 教学操作系统通常运行在资源受限的环境中, 预先加载可以减少运行时开销。
3. 启动速度: 预先加载可以加快系统启动速度, 提高实验效率。